

Go. Chapter VII: Interfaces

NEDO-OFFSEC

TABLE OF CONTENT

I	Interfaces as Contracts	4
II	Interface Types	7
	1. Важные интерфейсы из пакета io.....	7
	2. Встраивание интерфейсов (Interface Embedding).....	7
	3. Комбинация стилей.....	7
	4. Правила встраивания.....	7
	5. Что важно запомнить.....	7
	6. Пример: fmt.Fprintf и io.Writer.....	7
III	Interface Satisfaction	8
	1. Тонкий момент: Получатель (Pointer vs Value).....	8
	2. Композиция интерфейсов.....	8
	3. Пустой интерфейс interface{ }.....	8
IV	flag.Value	9
	1. Суть.....	9
	2. Пример: Свой флаг для температуры.....	9
	a. Как это работает.....	9
	b. Функция-конструктор.....	9
	c. Использование.....	9
	d. Запуск.....	9
	3. Ключевая мысль.....	9
V	Interface Values	10
	1. Устройство интерфейса.....	10
	2. Нулевое значение интерфейса (nil).....	10
	3. Присваивание конкретного типа интерфейсу.....	10
	4. Присваивание другого типа.....	10
	5. Значение интерфейса равно nil только когда оба компонента равны nil... ..	10
	6. Сравнение интерфейсов.....	10
	7. Как узнать тип внутри интерфейса.....	10
	8. Итог.....	10
VI	Sorting with sort.Interface	11
	1. Суть.....	11
	2. Как это работает.....	11
	3. Пример: срез строк.....	11
	4. Сортировка структуры.....	11
	5. Обратная сортировка.....	11
	6. Многоуровневая сортировка.....	11
	7. Готовые функции.....	12
	8. Ключевая мысль.....	12
VII	HTTP Handlers	13
	1. Суть.....	13
	2. Простейший сервер.....	13
	3. Маршрутизация по URL.....	13
	4. Использование ServeMux (мультиплексор).....	13
	5. Функция как обработчик (http.HandlerFunc).....	13

	6. Глобальный DefaultServeMux.....	14
	7. Важно!.....	14
	8. Ключевая мысль.....	14
VIII	Error Interface.....	15
	1. Суть.....	15
	2. Как создать ошибку.....	15
	3. Почему там указатель, а не значение?.....	15
	4. Форматирование ошибок.....	15
	5. Пример: системная ошибка.....	15
	6. Ключевая мысль.....	15
IX	Type Assertions.....	16
	1. Суть.....	16
	2. Два варианта.....	16
	а. Вариант 1: Конкретный тип.....	16
	б. Вариант 2: Интерфейс.....	16
	3. Безопасная форма (с проверкой).....	16
	4. Пример: распознавание ошибок.....	16
	5. Ключевая мысль.....	16
X	Type Switches.....	17
	1. Два стиля интерфейсов.....	17
	2. Пример: sqlQuote.....	17
	3. Синтаксис.....	17
	4. Как это работает.....	17
	5. Порядок важен.....	17
	6. Где используется.....	17
	7. Ключевая мысль.....	18

Go. Chapter VII: Interfaces

Интерфейсные типы выражают обобщения или абстракции поведения других типов. С помощью обобщения интерфейсы позволяют писать более гибкие и адаптируемые функции, не привязанные к деталям одной конкретной реализации.

Interfaces as Contracts

В Go есть еще одна разновидность типов – тип интерфейса. Интерфейс является абстрактным типом. Он не раскрывает представление или внутреннюю структуру своих значений, или набор основных поддерживаемых ими операций.

Он показывает только некоторые из их методов.

Если у вас есть значение интерфейсного типа, вы **НИЧЕГО** не знаете о том, чем оно **ЯВЛЯЕТСЯ**, вы знаете **ТОЛЬКО** то, что оно может **ДЕЛАТЬ** или, точнее какое поведение предоставляется его методами

ПРИМЕР:

У нас есть 2 функции:

- `fmt.Printf`, которая записывает результаты в стандартный поток вывода
- `fmt.Sprintf`, которая возвращает результат в виде строки `string`

Обе функции являются, по сути, **ОБЕРТКАМИ** для функции `fmt.Fprintf`, которая **НИЧЕГО НЕ ЗНАЕТ** о том, что будет с результатом, который она вычисляет.

```
package fmt

func Fprintf(w io.Writer, format string, args ...interface{}) (int, error)

func Printf(format string, args ...interface{}) (int, error) {
    return Fprintf(os.Stdout, format, args...)
}

func Sprintf(format string, args ...interface{}) string {
    var buf bytes.Buffer
    Fprintf(&buf, format, args...)
    return buf.String()
}
```

Первый параметр `Fprintf` вообще не является файлом, его тип – `io.Writer`, который является типом интерфейса со следующим объявлением:

```
package io

type Writer interface {
    // Write записывает len(p) байтов из p в базовый поток данных.
    // Метод возвращает количество байтов, записанных из p
    // (0 <= n <= len(p)), а любая ошибка вызывает прекращение записи.
    // Write должен вернуть ненулевую ошибку при n < len(p).
    // Write не должен изменять данные среза, даже временно.
    //
    // Реализации не должны сохранять p.
    Write(p []byte) (n int, err error)
}
```

Интерфейс `io.Writer` определяет контракт между `Fprintf` и вызывающими его функциями. С одной стороны, контракт требует, чтобы вызывающая функция предоставляла значение конкретного типа наподобие `*os.File` или `*bytes.Buffer`, который содержит метод с именем `Write` с соотв. сигнатурой и поведением.

Таким образом, нам неважно что за тип передан в `w`. Нам важно только то, что у этого типа есть метод `Write` в этом и фишка интерфейсов. Интерфейс `Writer` это такой тип, у которого есть функция `Writer`, описанная выше

Поэтому мы и могли передать несколько разных типов в `Fprintf` потому у него и есть обертки

ПРИМЕР:

```
type Dog struct {
    Name string
}

func (d Dog) Speak() string {
    return "Woof!"
}

type Cat struct {
    Name string
}

func (c Cat) Speak() string {
    return "Meow!"
}

type Speaker interface {
    Speak() string
}

func Greet(s Speaker) {
    fmt.Println("Hello,", s.Speak())
}

func main() {
    dog := Dog{Name: "Buddy"}
    cat := Cat{Name: "Kitty"}

    Greet(dog) // Hello, Woof!
    Greet(cat) // Hello, Meow!
}
```

СРАВНЕНИЕ GO vs CPP:

Golang

```
type Speaker interface { Speak() }
type Dog struct {}

func (d Dog) Speak() { fmt.Println("Woof!") }

var s Speaker = Dog{}
s.Speak() // ← Вызов через ITable (позднее связывание)
```

C++

```
class Animal {
public:
    virtual void Speak() = 0;
};

class Dog : public Animal {
public:
    void Speak() override { cout << "Woof!"; }
};

Animal* a = new Dog();
a->Speak(); // ← Вызов через VTable (позднее связывание)
```

Таким образом, интерфейсы Go очень похожи на виртуальные функции C++.

Отличие Go интерфейсов от виртуальных ф-й C++:

Они работают на схожем механизме позднего связывания, через таблицы методов. Но ключевое отличие в том, что реализация интерфейса в Go происходит неявно. Мне не нужно наследоваться от интерфейса (implements). Если у типа есть все методы, он автоматически удовлетворяет интерфейсу. Это делает код более гибким и свободным от жёстких иерархий

ВАЖНО:

Позднее связывание – это связывание, которое определяется в run-time

Раннее связывание – это связывание, которое определяется в compile-time

Interface Types

Важные интерфейсы из пакета io

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Closer interface {
    Close() error
}
```

Встраивание интерфейсов (Interface Embedding)

Интерфейсы можно встраивать друг в друга, как структуры.

```
type ReadWriter interface {
    Reader
    Writer
}

type ReadWriteCloser interface {
    Reader
    Writer
    Closer
}
```

Эквивалентно (без встраивания):

```
type ReadWriter interface {
    Read(p []byte) (n int, err error)
    Write(p []byte) (n int, err error)
}
```

Комбинация стилей

```
type ReadWriter interface {
    Read(p []byte) (n int, err error)
    Writer
}
```

Правила встраивания

Порядок методов не важен. Важно только их наличие.

Дублирование методов недопустимо. Если два встроенных интерфейса имеют методы с одинаковыми именами и сигнатурами, это нормально. Если сигнатуры разные – ошибка компиляции.

Встраивание работает только для интерфейсов. Встраивать структуры в интерфейсы **НЕЛЬЗЯ**.

Что важно запомнить

- Интерфейс определяет поведение.
- Конкретный тип реализует интерфейс неявно.
- Встраивание помогает строить сложные интерфейсы из простых.

Пример: fmt.Fprintf и io.Writer

```
func Fprintf(w io.Writer, format string, args ...interface{}) (int, error)

Fprintf не знает, что за тип передан в w:
```

- Это файл?
- Это буфер в памяти?
- Это сетевое соединение?
- Ему важно только одно: у типа есть метод Write.

Interface Satisfaction

Тип соответствует (удовлетворяет) интерфейсу, если он обладает всеми методами, которые требует интерфейс.

Правило присваиваемости интерфейсов очень простое:

выражение может быть присвоено интерфейсу, только если его тип соответствует этому интерфейсу.

Таким образом:

```
var w io.Writer
w = os.Stdout           // OK: *os.File имеет метод Write
w = new(bytes.Buffer)   // OK: *bytes.Buffer имеет метод Write
w = time.Second         // Ошибка: у time.Duration нет метода Write

var rwc io.ReadWriteCloser
rwc = os.Stdout         // OK: *os.File имеет методы Read, Write, Close
rwc = new(bytes.Buffer) // Ошибка: у *bytes.Buffer нет метода Close
```

Это правило применимо даже тогда, когда правая сторона сама по себе является интерфейсом:

```
w = rwc                 // OK: io.ReadWriteCloser имеет метод Write
rwc = w                 // Ошибка: io.Writer не имеет метода Close
```

Тонкий момент: Получатель (Pointer vs Value)

Метод с получателем-указателем (*T) и метод с получателем-значением (T) — это разные вещи.

Если метод объявлен как func (t *T) Method(), то интерфейсу удовлетворяет только *T.

Если метод объявлен как func (t T) Method(), то интерфейсу удовлетворяют и T, и *T.

ПРИМЕР:

```
type IntSet struct { /* ... */ }
func (*IntSet) String() string // метод с получателем-указателем

var s IntSet

var _ fmt.Stringer = &s // OK: *IntSet имеет метод String
var _ fmt.Stringer = s  // Ошибка: у IntSet нет метода String
```

Композиция интерфейсов

Если интерфейс A встроен в интерфейс B, то для удовлетворения B нужно реализовать все методы A и B.

```
go
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }
type ReadWriter interface {
    Reader
    Writer
}
```

Пустой интерфейс interface{}

interface{} не имеет методов. Ему удовлетворяет любой тип.

```
var any interface{}
any = true
any = 42
any = "hello"
any = map[string]int{"one": 1}
```


flag.Value

Суть

Пакет `flag` позволяет создавать свои собственные флаги для командной строки. Для этого нужно реализовать интерфейс `flag.Value`:

```
package flag

type Value interface {
    String() string
    Set(string) error
}
```

- `String()` – как отображать значение флага.
- `Set(string)` – как парсить строку, переданную в флаг, и сохранять значение.

Пример: Свой флаг для температуры

Программа позволяет передавать температуру в формате “100C” или “212F” и преобразует её в градусы Цельсия.

Как это работает

```
type celsiusFlag struct{ Celsius }           // Встраивание типа
func (f *celsiusFlag) Set(s string) error {
    var value float64
    var unit string
    fmt.Sscanf(s, "%f%s", &value, &unit)    // Парсинг числа и буквы
    switch unit {
    case "C", "°C":
        f.Celsius = Celsius(value)
    case "F", "°F":
        f.Celsius = FToC(Fahrenheit(value))
    default:
        return fmt.Errorf("неверная температура %q", s)
    }
    return nil
}
```

Функция-конструктор

```
func CelsiusFlag(name string, value Celsius, usage string) *Celsius {
    f := celsiusFlag{value}
    flag.CommandLine.Var(&f, name, usage)    // Регистрация в системе флагов
    return &f.Celsius
}
```

Использование

```
var temp = CelsiusFlag("temp", 20.0, "температура")
func main() {
    flag.Parse()
    fmt.Println(*temp)    // Автоматически вызовется String()
}
```

Запуск

```
$ ./tempflag -temp -18C
-18°C

$ ./tempflag -temp 212°F
100°C
```

Ключевая мысль

Интерфейс `flag.Value` даёт возможность расширить стандартную библиотеку Go, не меняя её код. Достаточно определить свой тип и реализовать два метода – `String()` и `Set()`.

Interface Values

Устройство интерфейса

Значение интерфейса в Go – это **ПАКЕТ ИЗ ДВУХ ЭЛЕМЕНТОВ**:

1. **Динамический тип** (конкретный тип, который хранится внутри).
2. **Динамическое значение** (само значение этого типа).

Оно как коробка, в которой лежит что-то конкретное, и мы знаем, что это за вещь.

Нулевое значение интерфейса (`nil`)

```
var w io.Writer // w == nil, тип и значение = nil
```

У такого интерфейса нет ни типа, ни значения. Если вызвать его метод – программа упадёт (паника).

Присваивание конкретного типа интерфейсу

```
w = os.Stdout
```

Теперь внутри `w`:

Тип: `*os.File`

Значение: указатель на `os.Stdout`

Вызов `w.Write([]byte("hello"))` превратится в вызов `(*os.File).Write`.

Присваивание другого типа

```
w = new(bytes.Buffer)
```

Теперь внутри `w`:

Тип: `*bytes.Buffer`

Значение: указатель на новый буфер

Вызов `w.Write([]byte("hello"))` теперь пишет в буфер, а не в консоль.

Значение интерфейса равно `nil` только когда оба компонента равны `nil`

```
var w io.Writer // w == nil (тип nil, значение nil)
```

```
w = (*bytes.Buffer)(nil) // w != nil (тип *bytes.Buffer, значение nil)
```

Важно! Это частая ошибка. Если интерфейсу присвоить `nil`-указатель, он перестаёт быть `nil` интерфейсом.

Сравнение интерфейсов

Два интерфейса равны (`==`), если:

Оба `nil`.

Или их динамические типы совпадают, и динамические значения равны.

Осторожно: Если динамический тип несравним (например, `[]int` или `map`), сравнение вызовет панику.

```
var x interface{} = []int{1, 2, 3}
fmt.Println(x == x) // Паника! Срезы несравнимы.
```

Как узнать тип внутри интерфейса

```
var w io.Writer = os.Stdout
fmt.Printf("%T\n", w) // "*os.File"
```

Итог

- Интерфейс = (тип, значение).
- `nil` интерфейс = (`nil`, `nil`).
- Интерфейс с `nil`-указателем `!= nil` интерфейсу.
- Сравнивай интерфейсы осторожно – внутри может быть несравнимый тип.

Sorting with sort.Interface

Суть

В Go сортировка работает через интерфейс `sort.Interface`.

```
package sort

type Interface interface {
    Len() int           // длина последовательности
    Less(i, j int) bool // сравнение элементов i и j
    Swap(i, j int)      // обмен элементов i и j
}
```

Как это работает

Любой тип, у которого есть эти 3 метода, можно отсортировать функцией `sort.Sort`.

Пример: срез строк

```
type StringSlice []string

func (p StringSlice) Len() int           { return len(p) }
func (p StringSlice) Less(i, j int) bool { return p[i] < p[j] }
func (p StringSlice) Swap(i, j int)      { p[i], p[j] = p[j], p[i] }
```

```
// Использование
names := []string{"Moby", "Delilah", "Alicia"}
sort.Sort(StringSlice(names))
```

Для строк уже есть готовый `sort.Strings(names)`.

Сортировка структуры

Структура `Track`:

```
type Track struct {
    Title  string
    Artist string
    Album  string
    Year   int
    Length time.Duration
}
```

Чтобы отсортировать срез треков по исполнителю:

```
type byArtist []*Track

func (x byArtist) Len() int           { return len(x) }
func (x byArtist) Less(i, j int) bool { return x[i].Artist < x[j].Artist }
func (x byArtist) Swap(i, j int)      { x[i], x[j] = x[j], x[i] }

// Сортировка
sort.Sort(byArtist(tracks))
```

Обратная сортировка

```
sort.Sort(sort.Reverse(byArtist(tracks)))
```

`sort.Reverse` оборачивает любой `sort.Interface` и меняет логику `Less` на противоположную.

Многоуровневая сортировка

```
type customSort struct {
    t    []*Track
    less func(x, y *Track) bool
}

func (x customSort) Len() int           { return len(x.t) }
func (x customSort) Less(i, j int) bool { return x.less(x.t[i], x.t[j]) }
func (x customSort) Swap(i, j int)      { x.t[i], x.t[j] = x.t[j], x.t[i] }
```

Сортировка по нескольким полям:

```
sort.Sort(customSort{tracks, func(x, y *Track) bool {  
    if x.Title != y.Title {  
        return x.Title < y.Title  
    }  
    if x.Year != y.Year {  
        return x.Year < y.Year  
    }  
    return x.Length < y.Length  
}})
```

Готовые функции

Пакет sort уже умеет сортировать:

```
sort.Ints([]int{3, 1, 4})           // []int  
sort.Strings([]string{"b", "a"})    // []string  
sort.Float64s([]float64{3.14, 1.0}) // []float64
```

Ключевая мысль

sort.Interface – это абстракция. Она работает с любой последовательностью: срез, массив, структура.

Главное – реализовать 3 метода: Len, Less, Swap.

HTTP Handlers

Суть

В Go веб-сервер строится на интерфейсе `http.Handler`:

```
package http

type Handler interface {
    ServeHTTP(w ResponseWriter, r *Request)
}
```

Любой тип, у которого есть метод `ServeHTTP`, является обработчиком HTTP-запросов.

Простейший сервер

```
type database map[string]float64

func (db database) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    for item, price := range db {
        fmt.Fprintf(w, "%s: %.2f\n", item, price)
    }
}

func main() {
    db := database{"shoes": 50, "socks": 5}
    log.Fatal(http.ListenAndServe("localhost:8000", db))
}
```

Сервер принимает любой запрос и выводит все товары.

Маршрутизация по URL

```
func (db database) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    switch r.URL.Path {
    case "/list":
        for item, price := range db {
            fmt.Fprintf(w, "%s: %.2f\n", item, price)
        }
    case "/price":
        item := r.URL.Query().Get("item")
        price, ok := db[item]
        if !ok {
            http.Error(w, "not found", http.StatusNotFound)
            return
        }
        fmt.Fprintf(w, "%.2f\n", price)
    default:
        http.Error(w, "not found", http.StatusNotFound)
    }
}
```

Использование ServeMux (мультиплексор)

```
func main() {
    db := database{"shoes": 50, "socks": 5}
    mux := http.NewServeMux()
    mux.Handle("/list", http.HandlerFunc(db.list))
    mux.Handle("/price", http.HandlerFunc(db.price))
    log.Fatal(http.ListenAndServe("localhost:8000", mux))
}
```

`ServeMux` – это роутер. Он сопоставляет URL с обработчиками.

Функция как обработчик (`http.HandlerFunc`)

```
type HandlerFunc func(w ResponseWriter, r *Request)

func (f HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) {
    f(w, r)
}
```

``HandlerFunc`` -- это адаптер. Любая функция с сигнатурой ``func(w http.ResponseWriter, r *http.Request)`` может стать ``http.Handler`` через преобразование типа.

```
mux.Handle("/list", http.HandlerFunc(db.list)) // явно
mux.HandleFunc("/list", db.list)              // сокращённо
```

Глобальный DefaultServeMux

```
func main() {
    db := database{"shoes": 50, "socks": 5}
    http.HandleFunc("/list", db.list) // регистрирует в глобальном роутере
    http.HandleFunc("/price", db.price)
    log.Fatal(http.ListenAndServe("localhost:8000", nil)) // nil = DefaultServeMux
}
```

Важно!

Каждый HTTP-запрос обрабатывается в своей go-подпрограмме. Если обработчик меняет общие данные – используйте `sync.Mutex`, чтобы избежать гонок.

Ключевая мысль

Интерфейсы `http.Handler` и `ServeMux` делают веб-серверы гибкими. Можно регистрировать любые обработчики, не меняя структуру сервера.

Error Interface

Суть

В Go ошибка – это не магия. Это просто интерфейс:

```
type error interface {  
    Error() string  
}
```

Любой тип, у которого есть метод `Error() string`, автоматически является ошибкой.

Как создать ошибку

```
import "errors"
```

```
err := errors.New("something went wrong")
```

Внутри `errors.New` возвращает указатель на структуру `errorString`, которая реализует `error`.

```
package errors
```

```
type errorString struct { text string }
```

```
func (e *errorString) Error() string { return e.text }
```

Почему там указатель, а не значение?

Чтобы две ошибки с одинаковым текстом не были равны:

```
fmt.Println(errors.New("EOF") == errors.New("EOF")) // false
```

Если бы использовалась структура, а не указатель, они могли бы быть равны, и это нарушило бы логику работы с `io.EOF`.

Форматирование ошибок

```
import "fmt"
```

```
err := fmt.Errorf("file %s not found", filename)
```

`fmt.Errorf` – это обёртка над `errors.New`, которая сначала форматирует строку.

Пример: системная ошибка

```
package syscall
```

```
type Errno uintptr
```

```
var errors = [...]string{  
    1: "operation not permitted",  
    2: "no such file or directory",  
}
```

```
func (e Errno) Error() string {  
    if int(e) < len(errors) {  
        return errors[e]  
    }  
    return fmt.Sprintf("errno %d", e)  
}
```

Теперь `syscall.Errno(2)` – это ошибка с текстом “no such file or directory”.

Ключевая мысль

Интерфейс `error` – это просто контракт на метод `Error()`. Это даёт гибкость: можно возвращать ошибки разных типов, но работать с ними через один интерфейс.

Type Assertions

Суть

Декларация типа (type assertion) — это операция, которая извлекает значение из интерфейса.

Синтаксис: `x.(T)`

- `x` — значение интерфейсного типа
- `T` — декларируемый тип

Два варианта

Вариант 1: Конкретный тип

Если `T` — конкретный тип, то `x.(T)` проверяет, хранит ли `x` именно этот тип.

```
var w io.Writer
w = os.Stdout

f := w.(*os.File)      // OK: f == os.Stdout
c := w.(*bytes.Buffer) // Паника: интерфейс хранит *os.File, а не *bytes.Buffer
```

Вариант 2: Интерфейс

Если `T` — интерфейс, то `x.(T)` проверяет, реализует ли динамический тип `x` интерфейс `T`.

```
var w io.Writer = os.Stdout
rw := w.(io.ReadWriter) // OK: *os.File реализует io.ReadWriter
```

Безопасная форма (с проверкой)

```
f, ok := w.(*os.File)
if ok {
    // f — *os.File, можно использовать
} else {
    // w хранит не *os.File
}
```

Если декларация неудачна, `ok == false`, а значение - нулевое для этого типа.

Пример: распознавание ошибок

Пакет `os` определяет структуру `PathError`:

```
type PathError struct {
    Op    string
    Path  string
    Err   error
}

func (e *PathError) Error() string {
    return e.Op + " " + e.Path + ": " + e.Err.Error()
}
```

Функция `os.IsNotExist` использует декларацию типа для проверки:

```
func IsNotExist(err error) bool {
    if pe, ok := err.(*PathError); ok {
        err = pe.Err
    }
    return err == syscall.ENOENT || err == ErrNotExist
}
```

Теперь можно отличить “файл не найден” от других ошибок.

```
err := os.Open("/no/such/file")
fmt.Println(os.IsNotExist(err)) // true
```

Ключевая мысль

Декларация типа (`x.(T)`) — это способ заглянуть внутрь интерфейса и получить конкретное значение или проверить, какой тип там хранится. Используй безопасную форму с `ok`, чтобы избежать паники.

Type Switches

Два стиля интерфейсов

1. **Полиморфизм подтипов** – интерфейс определяет поведение (методы). Примеры: `io.Reader`, `http.Handler`, `sort.Interface`.
2. **Распознаваемое объединение** – интерфейс хранит один из множества типов. Акцент на том, **какой** это тип, а не на методах.

Пример: sqlQuote

Функция преобразует значение любого типа в SQL-литерал:

```
func sqlQuote(x interface{}) string {
    switch x := x.(type) {
    case nil:
        return "NULL"
    case int, uint:
        return fmt.Sprintf("%d", x)    // x – interface{}, но мы знаем, что это число
    case bool:
        if x {
            return "TRUE"
        }
        return "FALSE"
    case string:
        return sqlQuoteString(x)      // x – string
    default:
        panic(fmt.Sprintf("unexpected type %T: %v", x, x))
    }
}
```

Синтаксис

```
switch x := x.(type) {
case nil:
    // x – interface{} (nil)
case int:
    // x – int
case string:
    // x – string
case bool, uint:
    // x – interface{} (несколько типов в одном case)
default:
    // x – interface{}
}
```

Как это работает

`x.(type)` – это специальная форма для `switch`.

В каждом `case` переменная `x` имеет конкретный тип этого `case`.

Если в одном `case` несколько типов (например, `int`, `uint`), то `x` остаётся `interface{}`.

Порядок важен

Если один тип является интерфейсом, он может “поглотить” другие. Например:

```
switch x.(type) {
case fmt.Stringer:
    // ...
case string:
    // ...
}
```

Здесь `string` никогда не выполнится, потому что `string` реализует `fmt.Stringer`.

Где используется

Парсинг JSON/YAML.

Обработка SQL-запросов (как в примере).

Логирование значений разных типов.

Работа с `interface{}` в обобщённых функциях.

Ключевая мысль

Выбор типа – это способ заглянуть внутрь `interface{}` и сделать разную логику для разных типов. Это как `switch`, но не по значению, а по типу.